

UD3 + UD4 – Examen (Modelo B)

POR FAVOR, LEE LAS INSTRUCCIONES ATENTAMENTE. DEBERÁS ENTREGAR ESTE MISMO DOCUMENTO EN FORMATO PDF (**CUALQUIER OTRO FORMATO DISTINTO DE PDF NO SERÁ CORREGIDO**). SE ESPERA QUE PUEDAS RESOLVER ESTAS ACTIVIDADES EMPLEANDO EXCLUSIVAMENTE TUS CONOCIMIENTOS Y LOS MATERIALES QUE HAYAS DESCARGADO PREVIAMENTE. EN CASO DE CUALQUIER EVIDENCIA DE COPIA, EL EXAMEN SE EVALUARÁ COMO SUSPENSO (0).

Pregunta 1. Crea dos redes virtuales en Docker (asignando IPs de tu elección). Despliega en cada uno de ellos una web sencilla empleando un servidor web de tu elección. Asegúrate de cumplir las siguientes premisas:

- Las subredes han de llamarse “Este” y “Oeste”.
- Las máquinas han de llamarse “Valencia” y “Lisboa”.

Para completar este ejercicio, deberás proporcionar lo siguiente:

1. Muestra los comandos necesarios para levantar ambos contenedores y sus correspondientes subredes, explicando brevemente los pasos a seguir, así como el estado del panel de control de Docker con ambos contenedores en ejecución. (2 puntos)

Para conseguir dos subredes abrimos un terminal en mi caso es bash y ejecutamos estos dos comandos para crearlos:

```
daw2@COMPUS040 MINGW64 ~  
$ docker network create --subnet 192.20.0.0/24 exam-net  
c6883899e41e6d1500478b549a8d9273e03bf97b0b6bb24d2d38f2adebc65410
```

```
daw2@COMPUS040 MINGW64 ~  
$ docker network create --subnet 192.21.0.0/24 exam-network  
c6328727d7dded3117a240fa35d913c3f1bf135ea1b84d17cd485a4c7d16d4bd
```

```
daw2@COMPUS040 MINGW64 ~
$ docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
e1cb56caebf1        bridge              bridge              local
c6883899e41e        exam-net            bridge              local
c6328727d7dd        exam-network        bridge              local
c22b17326617        host                host                local
bdcba9a294a         none                null                local
7037271a7d6a        private-net         bridge              local
cdfdba8a1556        proyecto-root_default bridge              local
51580b62e583        public-net          bridge              local
```

Con “docker network ls” podemos ver todas las subredes que tenemos, lo que hemos creado exam-net y exam-network

Ya tenemos las dos redes creadas, ahora tenemos que crear las maquinas(contenedores) con los siguientes comandos.

```
daw2@COMPUS040 MINGW64 ~
$ docker run -d --name Valencia --network exam-net nginx
3630c675557be1d4835a5c2871596d214c9a367952f4a184b1add60eb32b44db

daw2@COMPUS040 MINGW64 ~
$ docker run -d --name Lisboa --network exam-network nginx
467a529ce58f666252651cb843966f8006a00fe0081b09b73a4b477ea57afefb
```

Como utilizo el nginx antes en mi maquina ya tiene desagrada las imágenes para poder utilizarle.

☐	Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
☐	Valencia	3630c675557b	nginx		0%	2 minutes ago	   
☐	Lisboa	467a529ce58f	nginx		0%	1 minute ago	   

En esta captura podemos ver dentro de Docker Dekstop que se han creado bien y están en ejecución.

- ¿Es posible la comunicación entre ambos contenedores? Muestra pruebas que apoyen tu respuesta, argumentando brevemente tu respuesta. (1 punto)

Para hacer las pruebas de conexión entre contenedores tenemos que instalar el ping con el comando: `docker exec -it Valencia sh -> apt install -y iputils-ping.`

No puedo enseñar las pruebas como no puedo instalar el ping dentro de docker por falta de red.

Pero lo puedo argumentar que las redes no se pueden comunicarse entre si porque están en subredes distintos, pero si queremos hacer la comunicación usaremos el comando: “docker network connect exam-net Valencia” (¡dentro de Lisboa!) así metemos o mejor decir copiamos el contenedor de Lisboa dentro de la red de Valencia y se pueden comunicar.

3. Explica brevemente la diferencia entre una máquina virtual y un contenedor. Cita al menos una ventaja que implique el uso de contenedores frente al uso de máquinas virtuales. (1 punto)

Máquina virtual puede llevar onecesitar mucho más recursos y tiempo para montarle además de añadir/crear dentro por ejemplo el back que puede alimentar el front desde allí. En cunado un contenedor es mucho más ligero y fácil de hacer ideal para pruebas y ejecución en la maquina física.

Si necesitamos rapidez y confianza el uso de contendores es mejor al frente de maquinas virtuales.

Pregunta 2. Crea una orquestación local de contenedores empleando Docker Compose. Asegúrate de que tu proyecto cumple las siguientes premisas:

- Será necesario contar con dos contenedores:
 - Un contenedor actuará como servidor web. **Importante, no puedes utilizar Nginx** y se aconseja el uso de Apache o similar.
 - Un segundo contenedor actuará como front end. Puedes utilizar la tecnología de tu elección.
- El proyecto deberá mostrar una web de un negocio que puede ser ficticio, resaltando al menos 3 servicios ofrecidos, 2 ubicaciones y una breve presentación. Se valorará el uso de hoja de estilos o similar.

Para completar este ejercicio, deberás proporcionar lo siguiente:

1. El contenido de tu fichero docker-compose correctamente indentado para la orquestación local de los contenedores, explicando brevemente la funcionalidad lograda. (2 puntos)

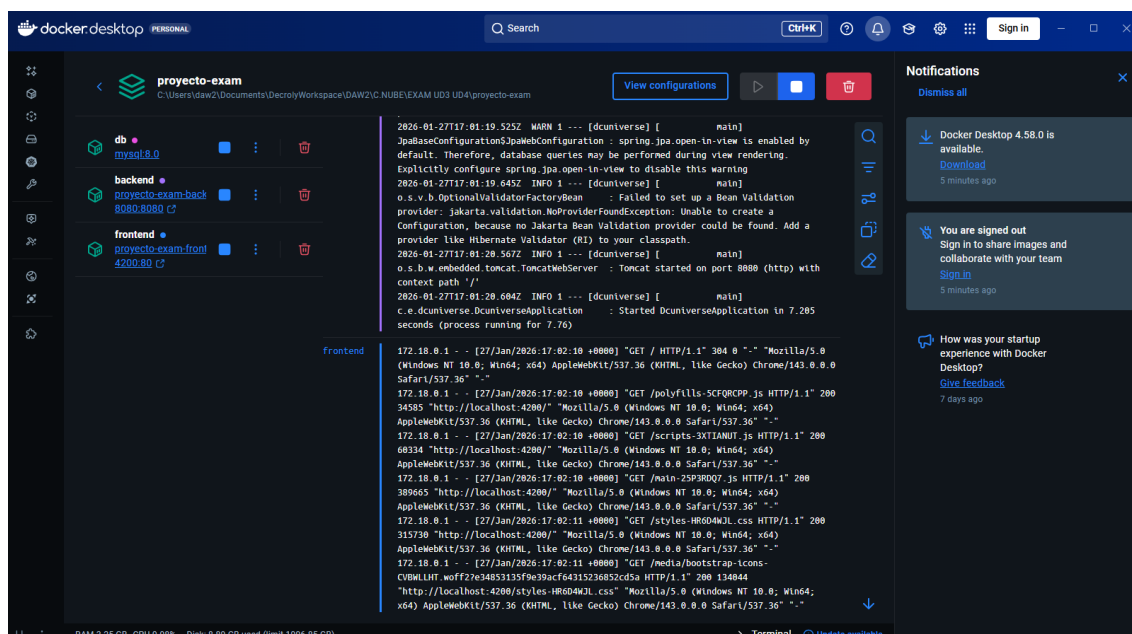
Lo primero es la estructura de carpetas

```

proyecto-exam
  o frontend
    ▪ Dockerfile
    ▪ (archivos front)
  o backend
    ▪ Dockerfile
    ▪ (archivos springboot)
  o docker-compose.yml
  
```

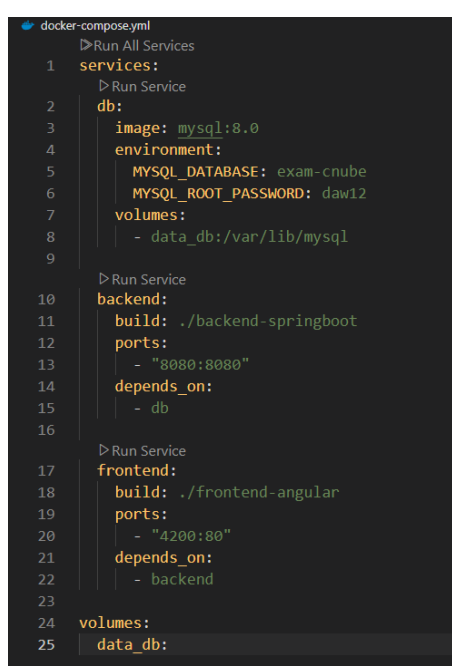
Como en la práctica de Docker Compose utilizo otro método de crearle lo voy enseñar este mismo, es otro proyecto, pero con mismos pasos para completar esta parte de la práctica.

Después de tener esta estructura tenemos que configurar los ficheros “dockerfile”. Luego de configurar los ficheros arrancamos/creamos el compose con el comando “docker-compose up” dentro de “proyecto-exam” (se necesita red para esto).



Dentro de nuestro Compose podemos ver nuestros tres contenedores de base de datos, backend y frontend. Al arrancar el Compose se levanta todos los tres contenedores a la vez.

Primer fichero “docker-compose.yml”



Lo primero que hacemos es agrupar todos los servicios, luego en el **bloque “db”** descargamos la imagen de mysql, nombre de la base de datos con contraseña de root, y al final la parte donde se guarda todos los datos para tener persistencia de datos. **Bloque “backend”**, es le decimos que utilice el back creado con el puerto correspondiente, y que espere la base de datos antes de arrancar, al final el **bloque “frontend”** es lo mismo que de back pero con su puerto y que espere al back antes de arrancar.

2. El contenido de los ficheros de configuración de los contenedores creados, explicando brevemente la funcionalidad lograda. (2 puntos)

Dockerfile de front

```
frontend-angular > dockerfile > ...  
1 FROM node:latest AS build  
2 WORKDIR /app  
3 COPY . .  
4 RUN npm install && npm run build  
5  
6 FROM nginx:alpine  
7 COPY --from=build /app/dist/*/browser /usr/share/nginx/html  
8 EXPOSE 80
```

Uso de Node.js solo para instalar las librerías y "traducir" mi código Angular a archivos que cualquier navegador entienda (HTML y JS). Y paso archivos limpios a un servidor Nginx, que es mucho más rápido y ligero. Así, el contenedor final ocupa poco espacio.

Dockerfile de back

```
backend-springboot > dockerfile > ...  
1 FROM eclipse-temurin:21-jdk-alpine  
2 COPY build/libs/*.jar app.jar  
3 ENTRYPOINT ["java", "-jar", "/app.jar"]
```

Usamos la versión de java 21, copiamos el archivo .jar dentro del contenedor, y el entrypoint para que se ejecute la aplicación automáticamente después de arrancar el contenedor.

3. Capturas de pantalla de tu proyecto en funcionamiento (barra de direcciones, contenido web, cuadro de mando de Docker). (2 puntos)